ОТЧЁТ

по лабораторной работе №1

«Основы программирования на Python»

Вариант 20. Доставка заказов

1. Цель работы

Получить практические навыки разработки программ на языке Python и освоить базовые конструкции языка: переменные и константы; основные типы данных; операции над данными; ввод и вывод информации; условные конструкции; циклы; функции; преобразование типов; работу со строками и числовыми данными. В процессе выполнения лабораторной работы необходимо разработать консольное приложение для решения индивидуальной задачи.

2. Постановка индивидуального задания

Вариант 20. Доставка заказов. Разработать программу анализа работы службы доставки. Для каждого заказа вводится расстояние доставки и время выполнения заказа. Программа должна:

– определить среднее время доставки;

– найти самый быстрый и самый медленный заказ;

– определить среднюю скорость доставки;

– определить количество заказов, выполненных позже нормативного времени;

– определить общий километраж.

Согласно общим требованиям методических указаний, программа запрашивает исходные данные у пользователя, проверяет корректность ввода,

выполняет вычисления, использует условные конструкции и циклы, содержит не менее двух собственных функций и выводит результаты в понятном виде. Базы данных, сторонние библиотеки, файлы и графический интерфейс не используются.

Для выполнения дополнительного задания реализованы меню приложения, повторный запуск операций без перезапуска программы, обработка нескольких наборов данных, дополнительные статистические показатели и более подробная проверка пользовательского ввода.

3. Краткие теоретические сведения

Python — интерпретируемый язык программирования высокого уровня с динамической типизацией. Тип переменной определяется автоматически по присвоенному значению, а узнать его можно функцией type().

Основные типы данных, использованные в работе: int (целые числа), float (вещественные числа), bool (логические значения True и False) и str (строки). Над числами выполняются операции +, -, *, /, //, %, **. Оператор / даёт вещественный результат, // — целую часть от деления, % — остаток.

Для ввода данных используется функция input(), которая всегда возвращает строку. Поэтому для получения чисел необходимо преобразование типов: int() и float(). Вывод выполняется функцией print(); для подстановки значений в текст удобны f-строки.

Условная конструкция if / elif / else позволяет выбирать действия в зависимости от условия. Сложные условия формируются операторами and, or и not. Операторы сравнения: ==, !=, >, <, >=, <=.

Цикл for используется для повторения действий заданное число раз, обычно вместе с range(). Цикл while выполняется, пока условие истинно. Функция — именованный фрагмент программы; return возвращает результат. Конструкция

if __name__ == "__main__": задаёт точку входа и не запускает main() при импорте модуля.

4. Описание программы

Консольное приложение analiz_dostavki.py анализирует работу службы доставки. Данные можно ввести вручную или получить с помощью генератора случайных значений. Расстояние задаётся в километрах, время — в минутах, нормативное время доставки запрашивается у пользователя.

Программа построена по рекомендуемой в методичке структуре: набор функций вычислений, функция main() и точка входа if __name__ == "__main__":. Меню реализовано циклом while и конструкцией if / elif / else. Пока пользователь не выберет пункт «Выход», можно вводить данные, смотреть статистику и выполнять дополнительный анализ.

Основные функции: read_positive_int() и read_positive_float() проверяют ввод; calculate_average() считает среднее значение; find_extreme_indices() находит самый быстрый и самый медленный заказ; calculate_speed() переводит время в часы и вычисляет скорость в км/ч; count_late_orders() считает опоздания; calculate_total_distance() определяет общий километраж. Средняя скорость доставки считается по формуле «общий путь / общее время», что соответствует средней скорости всего маршрута.

Дополнительный анализ выводит среднее расстояние, суммарное время, минимальную и максимальную скорость, долю опозданий, текстовую оценку качества работы и разброс времени. Пункт меню «Начать новый набор данных» позволяет обработать несколько выборок за одну сессию и сравнить их.

Проверка данных выполняется без аварийного завершения программы: количество заказов должно быть целым положительным числом, расстояние и время — положительными, допускается ввод дробных чисел через точку или

запятую. Некорректный пункт меню и попытка посмотреть статистику без данных сопровождаются сообщениями об ошибке.

Таблица 1 – Пример исходных данных (ручной ввод)

Номер заказа	**Расстояние, км**	**Время, мин**	**Норматив, мин**
1	8,5	25	60
2	12,0	40	
3	20,0	90	
4	6,2	18	

5. Исходный код программы

Листинг 1 – Файл analiz_dostavki.py

```
"""
Лабораторная работа №1. Вариант 20.
Анализ работы службы доставки.
"""

import random
import sys

# Корректный вывод русских символов в консоли Windows
if sys.platform == "win32":
    sys.stdout.reconfigure(encoding="utf-8")
    sys.stderr.reconfigure(encoding="utf-8")


def read_line(prompt):
    """Читает строку с клавиатуры. В неинтерактивном режиме повторяет ввод в консоль."""
    if sys.stdin.isatty():
        return input(prompt).strip()
    raw = sys.stdin.readline()
    if raw == "":
        raise EOFError
    text = raw.rstrip("\r\n")
```

```
    print(f"{prompt}{text}")
    return text.strip()


def read_positive_int(prompt):
    """Считывает целое положительное число и повторяет запрос при ошибке."""
    while True:
        text = read_line(prompt)
        if text.isdigit() and int(text) > 0:
            return int(text)
        print("Ошибка: введите целое положительное число.")


def is_positive_number(text):
    """Проверяет, что строка содержит положительное вещественное число."""
    text = text.replace(",", ".", 1)
    if text.count(".") > 1:
        return False

    parts = text.split(".")
    if len(parts) == 1:
        return parts[0].isdigit() and float(text) > 0

    left = parts[0]
    right = parts[1]
    if left == "":
        left = "0"
    if right == "":
        return False
    if left.isdigit() and right.isdigit():
        return float(left + "." + right) > 0
    return False


def read_positive_float(prompt):
    """Считывает положительное вещественное число и повторяет запрос при ошибке."""
    while True:
        text = read_line(prompt)
        if is_positive_number(text):
            return float(text.replace(",", ".", 1))
        print("Ошибка: введите положительное число (например, 12.5).")


def calculate_average(values):
    """Возвращает среднее арифметическое списка чисел."""
    total = 0
    for value in values:
        total += value
    return total / len(values)


def find_extreme_indices(values):
    """Возвращает индексы наименьшего и наибольшего значений."""
    min_index = 0
    max_index = 0
    for i in range(1, len(values)):
        if values[i] < values[min_index]:
```

```
            min_index = i
        if values[i] > values[max_index]:
            max_index = i
    return min_index, max_index


def calculate_speed(distance, time_minutes):
    """Рассчитывает скорость доставки в км/ч."""
    time_hours = time_minutes / 60
    return distance / time_hours


def count_late_orders(times, time_limit):
    """Считает заказы, выполненные позже нормативного времени."""
    late_count = 0
    for time_value in times:
        if time_value > time_limit:
            late_count += 1
    return late_count


def calculate_total_distance(distances):
    """Возвращает общий километраж всех заказов."""
    total = 0
    for distance in distances:
        total += distance
    return total


def input_orders_manually():
    """Запрашивает у пользователя данные о заказах."""
    count = read_positive_int("Введите количество заказов: ")
    distances = []
    times = []

    for i in range(count):
        print(f"\nЗаказ {i + 1}:")
        distance = read_positive_float("  Расстояние доставки, км: ")
        time_value = read_positive_float("  Время выполнения, мин: ")
        distances.append(distance)
        times.append(time_value)

    time_limit = read_positive_float("\nВведите нормативное время доставки, мин: ")
    return distances, times, time_limit


def generate_orders():
    """Формирует набор заказов с помощью генератора случайных значений."""
    count = read_positive_int("Введите количество заказов для генерации: ")
    distances = []
    times = []

    print()
    for i in range(count):
        distance = round(random.uniform(3.0, 35.0), 1)
        time_value = round(random.uniform(20.0, 140.0), 1)
        distances.append(distance)
```

```
        times.append(time_value)
        print(f"Заказ {i + 1}: расстояние {distance:.1f} км, время {time_value:.1f} мин")

    time_limit = read_positive_float("\nВведите нормативное время доставки, мин: ")
    return distances, times, time_limit


def print_orders(distances, times):
    """Выводит исходные данные по каждому заказу."""
    print("\nИсходные данные по заказам:")
    print(f"{'№':<4}{'Расстояние, км':<18}{'Время, мин':<14}{'Скорость, км/ч'}")
    for i in range(len(distances)):
        speed = calculate_speed(distances[i], times[i])
        print(f"{i + 1:<4}{distances[i]:<18.2f}{times[i]:<14.2f}{speed:.2f}")


def print_main_statistics(distances, times, time_limit):
    """Выводит показатели, требуемые индивидуальным заданием."""
    print_orders(distances, times)

    average_time = calculate_average(times)
    fastest_index, slowest_index = find_extreme_indices(times)
    total_distance = calculate_total_distance(distances)
    total_time = 0
    for time_value in times:
        total_time += time_value
    average_speed = calculate_speed(total_distance, total_time)
    late_count = count_late_orders(times, time_limit)

    print("\n----- Основная статистика -----")
    print(f"Количество заказов: {len(distances)}")
    print(f"Нормативное время: {time_limit:.2f} мин")
    print(f"Среднее время доставки: {average_time:.2f} мин")
    print(
        f"Самый быстрый заказ: №{fastest_index + 1} "
        f"({times[fastest_index]:.2f} мин, {distances[fastest_index]:.2f} км)"
    )
    print(
        f"Самый медленный заказ: №{slowest_index + 1} "
        f"({times[slowest_index]:.2f} мин, {distances[slowest_index]:.2f} км)"
    )
    print(f"Средняя скорость доставки: {average_speed:.2f} км/ч")
    print(f"Заказов позже нормативного времени: {late_count}")
    print(f"Общий километраж: {total_distance:.2f} км")


def evaluate_quality(late_percent):
    """Возвращает текстовую оценку качества работы службы доставки."""
    if late_percent <= 10:
        return "высокая (почти все заказы в срок)"
    elif late_percent <= 25:
        return "средняя (есть небольшие опоздания)"
    elif late_percent <= 50:
        return "пониженная (значительная доля опозданий)"
    else:
        return "низкая (большинство заказов с опозданием)"
```

```
def print_extra_analysis(distances, times, time_limit, history):
    """Выводит дополнительные статистические показатели."""
    speeds = []
    for i in range(len(distances)):
        speeds.append(calculate_speed(distances[i], times[i]))

    average_time = calculate_average(times)
    average_distance = calculate_average(distances)
    average_of_speeds = calculate_average(speeds)
    min_speed_index, max_speed_index = find_extreme_indices(speeds)
    total_distance = calculate_total_distance(distances)
    late_count = count_late_orders(times, time_limit)
    late_percent = late_count / len(times) * 100

    faster_than_average = 0
    for time_value in times:
        if time_value < average_time:
            faster_than_average += 1

    total_time = 0
    for time_value in times:
        total_time += time_value

    print("\n----- Дополнительный анализ -----")
    print(f"Среднее расстояние: {average_distance:.2f} км")
    print(f"Суммарное время всех доставок: {total_time:.2f} мин")
    print(f"Минимальная скорость: {speeds[min_speed_index]:.2f} км/ч (заказ №{min_speed_index +
1})")
    print(f"Максимальная скорость: {speeds[max_speed_index]:.2f} км/ч (заказ №{max_speed_index
+ 1})")
    print(f"Средняя скорость по отдельным заказам: {average_of_speeds:.2f} км/ч")
    print(f"Заказов быстрее среднего времени: {faster_than_average}")
    print(f"Доля опозданий: {late_percent:.1f}%")
    print(f"Оценка качества работы: {evaluate_quality(late_percent)}")
    min_time_index, max_time_index = find_extreme_indices(times)
    print(f"Разброс времени доставки: {times[max_time_index] - times[min_time_index]:.2f} мин")

    records = []
    for item in history:
        records.append(item)
    records.append(make_history_record(distances, times, time_limit))

    if len(records) > 1:
        print(f"\nЗа текущую сессию обработано наборов данных: {len(records)}")
        print("Сравнение наборов:")
        print(f"{'Набор':<8}{'Заказов':<10}{'Ср.                                    время':<14}{'Ср.
скорость':<16}{'Километраж':<14}{'Опозданий'}")
        for i in range(len(records)):
            item = records[i]
            print(
                f"{i + 1:<8}{item[0]:<10}{item[1]:<14.2f}"
                f"{item[2]:<16.2f}{item[3]:<14.2f}{item[4]}"
            )


def make_history_record(distances, times, time_limit):
```

```
    """Формирует краткую запись о наборе данных для сравнения."""
    total_distance = calculate_total_distance(distances)
    total_time = 0
    for time_value in times:
        total_time += time_value
    average_time = calculate_average(times)
    average_speed = calculate_speed(total_distance, total_time)
    late_count = count_late_orders(times, time_limit)
    return (len(distances), average_time, average_speed, total_distance, late_count)


def print_menu():
    print()
    print("===== Анализ работы службы доставки =====")
    print()
    print("1. Ввести данные вручную")
    print("2. Сгенерировать данные случайным образом")
    print("3. Показать статистику")
    print("4. Выполнить дополнительный анализ")
    print("5. Начать новый набор данных")
    print("6. Выход")
    print()


def main():
    distances = []
    times = []
    time_limit = 0
    datasets_count = 0
    history = []

    while True:
        print_menu()
        choice = read_line("Выберите действие: ")

        if choice == "1":
            if len(distances) > 0:
                history.append(make_history_record(distances, times, time_limit))
                print("Предыдущий набор сохранён для сравнения.")
            distances, times, time_limit = input_orders_manually()
            datasets_count += 1
            print("Данные успешно сохранены.")

        elif choice == "2":
            if len(distances) > 0:
                history.append(make_history_record(distances, times, time_limit))
                print("Предыдущий набор сохранён для сравнения.")
            distances, times, time_limit = generate_orders()
            datasets_count += 1
            print("Случайные данные успешно сохранены.")

        elif choice == "3":
            if len(distances) == 0:
                print("Ошибка: сначала введите или сгенерируйте данные.")
            else:
                print_main_statistics(distances, times, time_limit)
```

```
        elif choice == "4":
            if len(distances) == 0:
                print("Ошибка: сначала введите или сгенерируйте данные.")
            else:
                print_extra_analysis(distances, times, time_limit, history)

        elif choice == "5":
            if len(distances) > 0:
                history.append(make_history_record(distances, times, time_limit))
                print(f"Текущий набор сохранён. Всего обработано наборов: {datasets_count}")
                distances = []
                times = []
                time_limit = 0
                print("Можно ввести или сгенерировать новый набор заказов.")
            else:
                print("Текущий набор пуст. Сначала введите данные.")

        elif choice == "6":
            print("Работа программы завершена.")
            break

        else:
            print("Ошибка: выберите пункт меню от 1 до 6.")


if __name__ == "__main__":
    main()
```

6. Примеры работы программы

Ниже приведены снимки экрана консольного приложения. Сначала отображается меню, затем проверяется некорректный ввод, после чего выполняются расчёты по индивидуальному заданию.

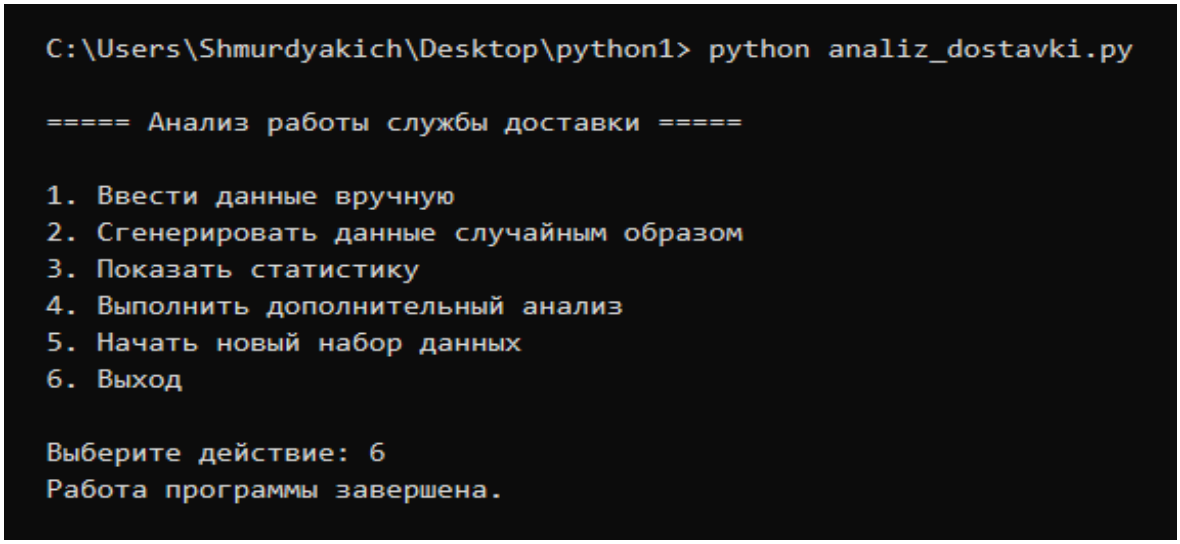


Рисунок 1 – Главное меню программы

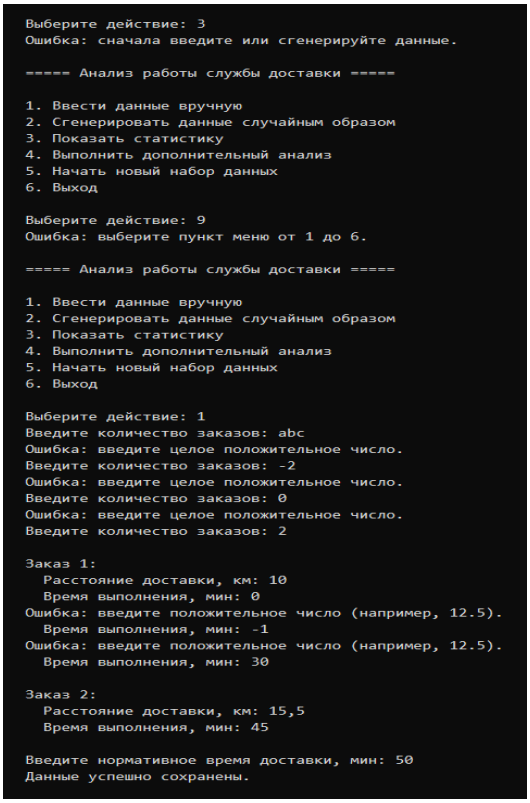


Рисунок 2 – Проверка корректности ввода и обработка ошибок

После отклонения некорректных значений (текст, ноль, отрицательные числа) программа приняла два заказа, в том числе расстояние 15,5 км, введённое через запятую.

```
Выберите действие: 1
Введите количество заказов: 4

Заказ 1:
  Расстояние доставки, км: 8.5
  Время выполнения, мин: 25

Заказ 2:
  Расстояние доставки, км: 12
  Время выполнения, мин: 40

Заказ 3:
  Расстояние доставки, км: 20
  Время выполнения, мин: 90

Заказ 4:
  Расстояние доставки, км: 6.2
  Время выполнения, мин: 18

Введите нормативное время доставки, мин: 60
Данные успешно сохранены.
```

Рисунок 3 – Ручной ввод четырёх заказов

```
Выберите действие: 3

Исходные данные по заказам:
№   Расстояние, км    Время, мин    Скорость, км/ч
1   8.50              25.00         20.40
2   12.00             40.00         18.00
3   20.00             90.00         13.33
4   6.20              18.00         20.67

----- Основная статистика -----
Количество заказов: 4
Нормативное время: 60.00 мин
Среднее время доставки: 43.25 мин
Самый быстрый заказ: №4 (18.00 мин, 6.20 км)
Самый медленный заказ: №3 (90.00 мин, 20.00 км)
Средняя скорость доставки: 16.20 км/ч
Заказов позже нормативного времени: 1
Общий километраж: 46.70 км
```

Рисунок 4 – Основная статистика по варианту 20

По данным таблицы 1 среднее время доставки составило 43,25 мин. Самый быстрый заказ — №4 (18 мин), самый медленный — №3 (90 мин). Средняя

скорость доставки равна 16,20 км/ч, позже норматива выполнен 1 заказ, общий километраж — 46,70 км.

```
Выберите действие: 4

----- Дополнительный анализ -----
Среднее расстояние: 11.68 км
Суммарное время всех доставок: 173.00 мин
Минимальная скорость: 13.33 км/ч (заказ №3)
Максимальная скорость: 20.67 км/ч (заказ №4)
Средняя скорость по отдельным заказам: 18.10 км/ч
Заказов быстрее среднего времени: 3
Доля опозданий: 25.0%
Оценка качества работы: средняя (есть небольшие опоздания)
Разброс времени доставки: 72.00 мин
```

Рисунок 5 – Дополнительный анализ текущего набора данных

```
Выберите действие: 2
Введите количество заказов для генерации: 5

Заказ 1: расстояние 32.7 км, время 114.2 мин
Заказ 2: расстояние 23.0 км, время 59.1 мин
Заказ 3: расстояние 30.5 км, время 74.6 мин
Заказ 4: расстояние 33.1 км, время 120.7 мин
Заказ 5: расстояние 29.2 км, время 86.2 мин

Введите нормативное время доставки, мин: 60
Случайные данные успешно сохранены.

===== Анализ работы службы доставки =====

1. Ввести данные вручную
2. Сгенерировать данные случайным образом
3. Показать статистику
4. Выполнить дополнительный анализ
5. Начать новый набор данных
6. Выход

Выберите действие: 3

Исходные данные по заказам:
№   Расстояние, км    Время, мин    Скорость, км/ч
1   32.70             114.20        17.18
2   23.00             59.10         23.35
3   30.50             74.60         24.53
4   33.10             120.70        16.45
5   29.20             86.20         20.32

----- Основная статистика -----
Количество заказов: 5
Нормативное время: 60.00 мин
Среднее время доставки: 90.96 мин
Самый быстрый заказ: №2 (59.10 мин, 23.00 км)
Самый медленный заказ: №4 (120.70 мин, 33.10 км)
Средняя скорость доставки: 19.59 км/ч
Заказов позже нормативного времени: 4
Общий километраж: 148.50 км
```

Рисунок 6 – Генерация случайных заказов и расчёт статистики

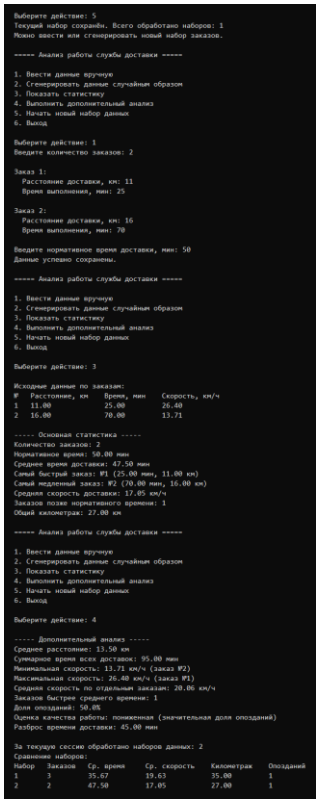


Рисунок 7 – Обработка нового набора данных и сравнение двух выборок

На рисунке 7 видно, что после сохранения первого набора можно ввести второй, повторно получить статистику и сравнить выборки по среднему времени, скорости, километражу и числу опозданий. Таким образом, приложение можно использовать несколько раз без перезапуска.

7. Ответы на контрольные вопросы

1. Что такое интерпретируемый язык программирования?

Интерпретируемый язык — это язык, программа на котором выполняется интерпретатором непосредственно по исходному коду (или по промежуточному байт-коду), без обязательной предварительной компиляции в машинный код отдельным этапом. Python относится к таким языкам: файл .py запускается интерпретатором, и команды выполняются последовательно.

2. Какие основные типы данных существуют в Python?

К базовым типам относятся целые числа int, вещественные числа float, логический тип bool и строки str. Кроме них часто используются последовательности и коллекции: список list, кортеж tuple, словарь dict, множество set. В данной лабораторной работе применялись int, float, bool, str и list.

3. Чем отличаются int и float?

Тип int хранит целые числа без дробной части, например 20 или 15. Тип float хранит числа с дробной частью, например 12.5 или 23.7. Результаты операций тоже различаются: 7 / 2 даёт 3.5 (float), а целочисленное деление применяется, когда нужна только целая часть.

4. Что возвращает функция input()?

Функция input() всегда возвращает строку типа str, даже если пользователь ввёл цифры. Поэтому после ввода числа «10» переменная содержит текст "10", а не целое число 10. Для расчётов строку нужно преобразовать.

5. Для чего используется int()?

Функция int() преобразует значение к целому числу. Её применяют, когда из строки ввода нужно получить количество элементов, номер пункта меню или другую целую величину. Если преобразование невозможно, возникает ошибка, поэтому в программе сначала проверяется, что введены цифры.

6. Чем отличаются операторы /, // и %?

Оператор / выполняет обычное деление и возвращает вещественный результат: 7 / 2 равно 3.5. Оператор // выполняет целочисленное деление и отбрасывает дробную часть: 7 // 2 равно 3. Оператор % возвращает остаток от деления: 7 % 2 равно 1.

7. Что такое логический тип bool?

Логический тип bool принимает два значения: True (истина) и False (ложь). Он используется в условиях: результат сравнения age >= 18 имеет тип bool и определяет, какая ветка if будет выполнена.

8. Какие операторы сравнения существуют в Python?

Основные операторы сравнения: == (равно), != (не равно), > (больше), < (меньше), >= (больше или равно), <= (меньше или равно). Они возвращают True или False и применяются в условных конструкциях и циклах while.

9. Для чего используется конструкция if/elif/else?

Конструкция if/elif/else нужна, чтобы программа выбирала одно из нескольких действий в зависимости от условия. Сначала проверяется if, при ложности — следующие elif, и если ни одно условие не выполнилось, выполняется else. В работе она используется в меню, при оценке качества доставки и при проверке данных.

10. Чем отличаются циклы for и while?

Цикл for удобен, когда известно количество повторений или нужно перебрать элементы последовательности, например range(count) или список заказов. Цикл while повторяет действия, пока условие истинно, и применяется, когда момент окончания заранее неизвестен, например пока пользователь не выберет выход из меню.

11. Для чего используется функция range()?

Функция range() создаёт последовательность целых чисел, которую обычно перебирают циклом for. Она позволяет повторить ввод или обработку заданное число раз, не дублируя команды вручную.

12. Что произойдёт при выполнении range(5)?

Будет построена последовательность 0, 1, 2, 3, 4 — пять чисел, начиная с нуля и не включая 5. Цикл for i in range(5) выполнится пять раз.

13. Что такое функция?

Функция — это именованный фрагмент программы, который решает отдельную задачу и может вызываться многократно. У функции есть имя, список параметров и тело. В лабораторной работе функции выделены для ввода, расчёта среднего, поиска минимума и максимума, подсчёта опозданий и вывода статистики.

14. Для чего используется return?

Оператор return завершает выполнение функции и возвращает результат в точку вызова. Например, calculate_average(values) возвращает среднее значение, которое затем можно вывести или использовать в других вычислениях.

15. Чем параметр функции отличается от возвращаемого значения?

Параметр — это входные данные, которые функция получает при вызове, например список времён доставки. Возвращаемое значение — это результат работы функции, который она отдаёт через return, например среднее время. Параметры «входят» в функцию, результат «выходит» из неё.

16. Для чего используется оператор and?

Оператор and объединяет условия по логическому «И»: составное условие истинно только тогда, когда истинны все его части. Пример: text.isdigit() and int(text) > 0 — строка состоит из цифр и число положительное.

17. Для чего используется оператор or?

Оператор or объединяет условия по логическому «ИЛИ»: достаточно, чтобы было истинно хотя бы одно из них. Его используют, когда действие нужно выполнить при любом из нескольких вариантов.

18. Что такое f-строка?

F-строка — это строковый литерал с префиксом f, в который можно подставлять значения переменных и выражений в фигурных скобках. Пример: f"Среднее время доставки: {average_time:.2f} мин". Такой способ вывода рекомендован методическими указаниями.

19. Зачем необходимо преобразование типов?

Преобразование типов нужно, потому что данные из input() приходят как строки, а арифметические операции выполняются над числами. Без int() и float() нельзя корректно сложить расстояния, найти среднее время или сравнить время с нормативом. Также преобразование позволяет округлять и форматировать результаты.

20. Для чего используется конструкция if __name__ == "__main__":?

Эта конструкция задаёт точку входа в программу. Код внутри неё выполняется только при прямом запуске файла, а не при импорте модуля в другую программу. В работе после этой проверки вызывается main(), как рекомендовано в методичке.

21. Что такое область видимости переменной?

Область видимости — это часть программы, в которой имя переменной доступно. Локальные переменные, созданные внутри функции, снаружи этой функции не видны. Переменные, заданные в main(), используются в main() и при необходимости передаются в другие функции как аргументы.

22. Что произойдёт при делении целого числа на ноль?

Интерпретатор Python вызовет исключение ZeroDivisionError, и программа остановится, если ошибка не обработана. Поэтому в работе время доставки проверяется на положительность: нулевое время не допускается, деления на ноль при расчёте скорости не происходит.

23. Чем = отличается от ==?

Оператор = выполняет присваивание: записывает значение в переменную (age = 20). Оператор == сравнивает два значения и возвращает True или False (age == 20). В условиях меню используется именно ==, иначе произошла бы попытка присваивания вместо проверки.

24. Зачем разбивать программу на несколько функций?

Разбиение на функции делает программу понятнее: каждая функция отвечает за одну операцию. Код не дублируется, его проще проверять и объяснять на защите. Методические указания прямо требуют не менее двух собственных функций и рекомендуют не писать всю программу одним блоком.

25. Какие преимущества имеет использование циклов по сравнению с многократным дублированием инструкций?

Цикл позволяет обработать любое количество заказов одной и той же последовательностью команд. При изменении числа элементов не нужно копировать строки ввода и расчётов. Программа становится короче, меньше риск ошибки, а логика обработки набора значений соответствует требованию индивидуального задания.

8. Вывод

В ходе лабораторной работы №1 разработано консольное приложение на языке Python для анализа работы службы доставки (вариант 20). Программа запрашивает расстояние и время по каждому заказу, проверяет корректность ввода, вычисляет среднее время доставки, находит самый быстрый и самый медленный заказ, определяет среднюю скорость, число опозданий относительно нормативного времени и общий километраж.

При реализации освоены переменные и типы данных, арифметические операции, ввод и вывод, преобразование типов, условные конструкции, циклы for и while, функции и f-строки. Программа структурирована через набор функций и точку входа if __name__ == "__main__":. Дополнительно

реализованы меню, повторные операции без перезапуска, генератор случайных данных, обработка нескольких наборов и расширенная статистика, что соответствует дополнительному заданию методических указаний.